

RSCTF TECHNICAL PRACTICE PAPER

HOW RSCTF SCORES KING OF THE HILL: CROWN CYCLES AND
NORMALIZED API ARENAS

Dimas Maulana

RSCTF Project · Competition Platform

Implementation-aligned manuscript · Version 3.0 · 28 July 2026

Scoring policy: one fixed formula per KotH format; no scoring versions

ABSTRACT

RSCTF implements two King of the Hill (KotH) formats with different evidence semantics. Marker KotH is an exclusive boot2root contest: one shared machine has at most one observed controller, and the fixed local score is $100R(0.25A + 0.55C + 0.20\sqrt{AC})$, where A is confirmed acquisition, C is sustained control, and R is service reliability during responsibility. API arena KotH is a multi-team application contest: every eligible team can produce evidence in the same tick. RSCTF normalizes challenge-native integer ratios into activity E , equal-weight objective performance P , and integrity I . It calculates a weighted harmonic core $B = 1/(0.35/E + 0.65/P)$, with $B = 0$ when either E or P is zero, and assigns tick score $100IB$. The platform calculates the nonlinear API result per tick before averaging, preventing evidence from different ticks from being recombined into performance that never occurred. Both formats use exact game, hill, round, cycle, reset, target, container, and capability-generation fences; an independent functional checker; field-wide void treatment for infrastructure failures; pristine crown-cycle replacements; bounded hill normalization; and finalized epoch settlement. The API referee submits signed evidence rather than points or raw capabilities. Omitted teams receive explicit zero rows, while missing, changing, late, or unhealthy field snapshots are void. These properties bound and audit the calculation, but they do not establish empirical fairness or prove that a challenge's evidence definitions measure the intended skill.

Keywords: King of the Hill; capture the flag; boot2root; API challenge; normalization; harmonic mean; anti-cheat; crown cycle; evidence integrity; RSCTF

Document status: technical-practice report aligned to the repository revision that contains it; not a claim of peer review or empirical validation. Appendix C maps the rules to source files.

START HERE: KOTH IN 90 SECONDS

Choose the format shown on the hill

Marker / boot2root means one shared machine and one holder:

1. exploit or administer the hill;
2. place your current capability in `/koth/king`;
3. keep the service healthy through confirmation; and
4. hold it until another team takes over or the clean reset begins.

API arena means every team may score concurrently:

1. use the challenge's player-facing API;
2. complete meaningful, verified actions;
3. optimize each published objective; and
4. avoid invalid attempts, which lower integrity.

Players never call the organizer's signed referee endpoint.

The two fixed formulas

Marker KotH:

```
Core = 0.25A + 0.55C + 0.20 * sqrt(A * C)
Local = 100 * R * Core
```

API arena KotH:

```
B = 0                                if E = 0 or P = 0
B = 1 / (0.35 / E + 0.65 / P)        otherwise
Tick = 100 * I * B
```

Marker **A/C/R** and arena **E/P/I** are different evidence, even though the board uses the same three compact metric columns. The hill's format badge determines their labels.

What resets and what settles

Several ticks form a **crown cycle**. At its boundary, RSCTF pauses the hill, destroys the old container, creates one pristine replacement from the frozen image, issues new capabilities, verifies readiness, and then resumes. Old capabilities, player changes, transient API sessions, and stale evidence stop working.

Several cycles form an **epoch**. **Projected** includes unfinished evidence; **Settled** includes only finalized epochs and determines official rank.

1. SCOPE AND DESIGN REQUIREMENTS

1.1 Why the formats are separate

A boot2root hill asks which team controls one shared target. Its natural evidence is exclusive identity, duration, and service responsibility. An application arena may instead expose puzzles, transactions, simulations, or protocol objectives that several teams can complete simultaneously. Treating the latter as a disguised one-holder marker discards useful evidence and encourages last-write races. Treating the former as a generic point feed gives an external component authority over the official score.

RSCTF therefore freezes one of two meanings for each hill:

Format	Concurrent scorers	Evidence	Platform result
Marker	At most one observed controller per tick	current capability, confirmation, control, checker verdict	A, C, R, marker formula
Api	Every eligible team	bounded activity, objective components, valid/all actions	E, P, I, arena formula

An API credential configured before official scoring selects **Api**; otherwise the hill is **Marker**. The frozen source cannot change during official scoring.

1.2 Shared invariants

Both formats must preserve the following requirements:

1. **Constant policy.** Each format has one formula; organizers cannot select a version or alter coefficients.
2. **Bounded result.** One hill-epoch score remains in **[0,100]**, and normalized hill aggregation preserves the 100-point epoch ceiling.
3. **Current evidence only.** Every scored sample belongs to the exact game, challenge, round, cycle, reset attempt, target, container, and capability generation.
4. **Independent health.** A functional checker, not the scoring feed, decides whether the shared application works.
5. **No stale carry.** Missing team evidence becomes zero in an otherwise valid API tick. A missing field snapshot becomes void; neither repeats a previous result.
6. **Infrastructure neutrality.** Reset, readiness, incomplete issuance, and platform-attributed failures do not become participant penalties.
7. **Auditable settlement.** Immutable tick evidence precedes epoch finalization, and retries cannot create duplicate credit.
8. **Bounded and frozen state.** Inputs, rosters, objective counts, replay memory, and retained challenge evidence have explicit limits. The first accepted snapshot containing a recognized team freezes the objective component count for the event. Referee credential rotation or revocation cannot change that challenge-owned scheme.

Marker-specific requirements are contestable exclusive control, qualified capture, and responsibility for service health. API-specific requirements are multi-team normalization, meaningful activity, same-tick integrity, and challenge evidence that requires actual play.

1.3 Relation to other systems

Bock, Hughey, and Levin describe an instructional KotH in which taking a shared machine also creates responsibility for critical services [2]. Marker KotH follows that exclusive-control interpretation while using periodic clean resets and fixed epoch weights.

CTFd documents a KotH agent that reports the current identifier and awards a configured reward at each check [3]. This is comparable to marker sampling, although RSCTF separately records provisional control, confirmation, duration, and reliability.

rCTF documents a signed dynamic-score endpoint that accepts absolute **userId/points** setters. Its contract permits negative points, preserves an omitted team's earlier value, and relies on the publisher to stop after the event deadline [5]. RSCTF reuses the useful exact-body HMAC and replay ideas, but its API arena does not accept external points. It applies event and round gates, normalizes bounded evidence, writes explicit zeros for omitted eligible teams, and calculates every score internally.

FAUST CTF is an Attack & Defense system rather than a shared hill, but it offers an operational comparison because it also uses repeated rounds and service checks [4]. Its target topology and score are not the RSCTF KotH model.

2. COMPETITION PROTOCOL

2.1 Marker / boot2root protocol

RSCTF runs one managed shared container for each marker hill. It issues a high-entropy bearer capability to every eligible team for the exact hill and reset attempt. A team claims by writing its current value to:

```
/koth/king
```

The checker reads the marker immediately before and after the functional probe. A stable eligible capability creates control and responsibility evidence. The first healthy observation is provisional; the configured consecutive healthy streak confirms acquisition. A different capability begins a different claim.

The previous cycle's team or tied teams with the most confirmed healthy controlled ticks normally receive the configured opening cooldown, unless blocking all tied leaders would leave no challenger. The blocked tick is removed from that team's personal opportunity denominator.

2.2 API arena protocol

The player-facing application receives a capability through its documented mechanic, hashes it immediately with SHA-256, and records only the lowercase digest alongside independently verified actions. The trusted referee runs outside the player-facing container.

For each active scoring round, the referee fetches a context bound to the exact runtime and receives the current eligible capability hashes. It filters the untrusted challenge feed against that set and submits:

- one activity **earned/possible** ratio;
- one to sixteen objective **earned/possible** ratios; and
- one integrity **valid/total** ratio per active team.

The signed body contains **tokenHash**, never the raw bearer token. It contains no team ID and no point value. RSCTF resolves hashes against current capabilities, normalizes the evidence, and reports submitted and recognized counts.

The checker allows a bounded six-second current-round arrival window, then reads the complete snapshot before and after its independent functional probe. A byte-equivalent current snapshot plus an **ok** checker verdict creates one dense result row for every frozen eligible team. Omitted teams receive zero. A changing snapshot or unhealthy shared application voids the field-wide tick.

API mode does not elect a holder, create a provisional crown, confirm acquisition, or apply champion-cooldown scoring. Multiple teams can score in the same tick.

2.3 Crown-cycle lifecycle

Both formats use a durable lifecycle:

```
finalize → pause → audit/snapshot → destroy → create → issue → readiness → resume
```

At every boundary, RSCTF stores the transition phase, prevents a second replica from owning the same hill operation, destroys the previous container, launches one replacement from the snapshotted image, clears transient claim or API snapshot state, revokes old capabilities, issues the new exact window, and resumes only after readiness and the functional checker succeed.

Marker mode also derives its previous champion and installs any enforceable cooldown. API mode has no exclusive champion; the lifecycle reset instead clears sessions, challenge state, stale hashes, and accumulated player changes.

Reset and readiness intervals create no scoring opportunity. A crash-safe retry resumes the stored phase rather than starting a parallel replacement.

2.4 Exact attribution and void evidence

One immutable checker result is allowed per game, challenge, and scoring round. Scorable evidence must match the frozen hill, active cycle, reset attempt, container identity, round deadline, and current capability window.

InternalError, incomplete token issuance, and uncertain runtime inspection are platform voids. A confirmed stopped container is handled through durable recovery. For marker mode, the responsible team may receive the final authoritative availability outcome before reset. For API mode, the shared application failure voids all teams for that tick because the checker cannot attribute a global service failure to one arena participant.

3. EVIDENCE AND SCORING METHOD

3.1 Marker acquisition, control, and reliability

For team i , hill h , and epoch e , define:

Symbol	Marker evidence
x_{ihe}	capability windows in which the team confirmed acquisition
y_{ihe}	windows in which the team had at least one eligible opportunity
u_{ihe}	scorable ticks controlled by the team's exact capability
s_{ihe}	personally eligible scorable ticks after void/cooldown removal
b_{ihe}	responsible ticks with checker verdict OK
d_{ihe}	ticks for which the team was responsible

The rates are:

$$(1) \frac{\frac{A_{ihe}}{x_{ihe}} \cdot \frac{y_{ihe}}{C_{ihe}} \cdot \frac{s_{ihe}}{R_{ihe}} \cdot \frac{b_{ihe}}{d_{ihe}}}{1}$$

An empty denominator produces zero. The fixed marker core and local score are:

$$(2) \frac{B_{ihe}^M}{0.25 \cdot \frac{A_{ihe}}{C_{ihe}} + 0.55 \cdot \frac{C_{ihe}}{A_{ihe}} + 0.20 \cdot \sqrt{A_{ihe} C_{ihe}}}$$

$$(3) \frac{L_{ihe}^M}{100 \cdot \frac{R_{ihe}}{B_{ihe}}}$$

Control has the largest direct coefficient. The geometric balance term rewards teams that both acquire and retain the hill, while reliability constrains the complete result.

3.2 API evidence normalization

For team i , hill h , tick t , let activity evidence be a_{iht}^+ / a_{iht}^* , integrity evidence be v_{iht} / n_{iht} , and let objective j report o_{ihtj}^+ / o_{ihtj}^* . Every numerator and denominator is an integer with:

$$(4) \frac{0 \leq \text{earned} \leq \text{possible} \leq 10^{12}}$$

RSCTF calculates:

$$(5) \frac{E_{iht}}{a_{iht}^+ \cdot \frac{a_{iht}^*}{v_{iht}} \cdot \frac{v_{iht}}{n_{iht}}}$$

$$(6) \frac{p_{ihtj}}{o_{ihtj}^+ \cdot \frac{o_{ihtj}^*}{p_{ihtj}} \cdot \frac{p_{ihtj}}{1} \cdot \frac{1}{\sum_{j=1}^m p_{ihtj}}} \leq \frac{1}{16}$$

Each objective is converted to a fixed integer normalization scale before the mean. This makes Equation (6) deterministic without trusting floating-point values in the signed request. More importantly, native scale does not imply weight: a **9/10** objective and a **900/1000** objective each contribute one normalized component.

3.3 API tick formula

The API core is a weighted harmonic mean:

$$(7) \frac{B_{iht}^A}{\begin{cases} 0, & E_{iht} = 0 \text{ or } P_{iht} = 0, \\ \frac{1}{0.35 / E_{iht} + 0.65 / P_{iht}}, & \text{otherwise.} \end{cases}}$$

The immutable tick score rate and point value are:

$$(8) \frac{G_{iht}}{\frac{I_{iht}}{B_{iht}^A} \cdot \frac{L_{iht}^A}{100} \cdot G_{iht}}$$

Both activity and objective performance are necessary. The larger objective weight makes poor objective performance more costly than equally poor activity, but the harmonic form remains sensitive to either bottleneck. Integrity multiplies the same tick's core.

For an epoch containing T_{he} scorable API ticks, RSCTF displays the mean channels but scores the mean of immutable tick results:

$$(9) \frac{\frac{\bar{E}_{ihe}}{1} \cdot \frac{1}{T_{he}} \sum_t E_{iht}^t \cdot \frac{\bar{P}_{ihe}}{1} \cdot \frac{1}{T_{he}} \sum_t P_{iht}^t \cdot \frac{\bar{L}_{ihe}}{1} \cdot \frac{1}{T_{he}} \sum_t L_{iht}^t}{\left(\frac{L_{ihe}^A}{100} \right) \left(\frac{1}{T_{he}} \sum_t G_{iht} \right)}$$

RSCTF does not insert the averages from Equation (9) back into Equation (7). That would permit temporal mixing. For example, perfect performance with zero integrity in one tick and perfect integrity with no play in another tick must remain two zero-score ticks.

3.4 Hill normalization

Before scoring starts, every hill receives a frozen weight w_h in $[0.8, 1.2]$. Let $z_{he} = 1$ when hill h has at least one field-wide scorable tick in epoch e , and zero otherwise. The normalized team epoch score is:

$$(11) \frac{\sum_{ie} E_{ie} \sum_h z_{he} w_h L_{ihe}}{\sum_h z_{he} w_h}$$

Here L_{ihe} is Equation (3) for a marker hill or Equation (10) for an API hill. A wholly void hill contributes no numerator or denominator. Once a hill has valid field evidence, omitted API teams retain explicit zero rows rather than removing that hill from their personal calculation.

3.5 Epoch weight, settlement, and rank

A complete evidence-bearing epoch has weight $q_e = 1$. If the event ends after r_e of the configured n scoring ticks in its final epoch:

$$(12) \frac{q_e}{\frac{r_e}{n}}$$

The event score is:

$$(13) \frac{T_i}{\sum_e q_e E_{ie}} \cdot \frac{\sum_e q_e E_{ie}}{\sum_e q_e}$$

There is no late-epoch multiplier. **Projected** includes open evidence; **Settled** includes finalized epochs only. Exact Settled ties use the format-neutral board columns in this order: second metric, third metric, first metric evidence count, and participation ID. Those columns mean control/reliability/acquisition for marker hills and objective/integrity/activity for API arenas.

3.6 Reading the scoreboard

The hill badge and column labels are part of the scoring contract:

Badge	First metric	Second metric	Third metric	Crown state
Marker	Acquisition	Control	Reliability	confirmed/prc holder
API arena	Activity	Objective	Integrity	none

Mixed games label aggregate columns with both meanings and preserve source-specific labels in each hill detail. Selecting the scoring information control shows both formulas and the no-carry rule.

4. WORKED EXAMPLES

4.1 Marker example

Suppose a team confirms one of two eligible acquisition windows, controls three of eight personally eligible ticks, and keeps the service healthy in four of five responsible ticks:

$$A = 1 / 2 = 0.5000, \quad C = 3 / 8 = 0.3750, \quad R = 4 / 5 = 0.8000$$

$$B^M = 0.25(0.5000) + 0.55(0.3750) + 0.20\sqrt{(0.5000)(0.3750)} \approx 0.417853,$$

$$L^M = 100(0.8000)(0.417853) \approx 33.43.$$

The team receives control evidence from the first stable eligible observation, but acquisition appears only after the healthy confirmation threshold.

4.2 API normalization example

Suppose one API tick reports:

```
{
  "activity": {"earned": 4, "possible": 5},
  "objectives": [
    {"earned": 7, "possible": 10},
    {"earned": 750, "possible": 1000}
  ],
  "integrity": {"earned": 19, "possible": 20}
}
```

The two objective scales are normalized independently:

$$E = 4 / 5 = 0.8000, \quad P = ((7 / 10) + (750 / 1000)) / 2 = 0.72$$

$$B^A = \frac{1}{0.35 / 0.8000 + 0.65 / 0.7250} \approx 0.749596.$$

$$L^A = 100(0.9500)(0.749596) \approx 71.21.$$

Adding zeros to the native throughput scale would not change its normalized share. Adding a duplicate objective entry would change the declared evidence model and is prohibited by the organizer guidelines as hidden weighting.

4.3 Why activity is mandatory

If $E = 0$, $P = 1$, and $I = 1$, Equation (7) sets the core to zero. A referee cannot grant points by reporting performance without verified activity. Likewise, $E = 1$ and $P = 0$ scores zero.

At $E = 0.2$, $P = 0.9$, and $I = 1$:

$$B^A = \frac{1}{0.35 / 0.2 + 0.65 / 0.9} \approx 0.404494,$$

so the tick scores approximately 40.45, not the 90 points suggested by the strong objective channel alone.

4.4 Same-tick integrity blocks temporal mixing

Consider two ticks:

Tick	E	P	I	Tick score
1	1	1	0	0
2	0	0	1	0

The displayed channel means are each **0.5**, but Equation (10) averages the two stored zero score rates and returns zero. Calculating a formula from the channel means would incorrectly create points for behavior that never had performance and integrity together.

4.5 Missing team and field void

If a valid snapshot contains evidence for 99 of 100 eligible teams, RSCTF writes the omitted team's rates as zero for that tick. If the referee submits no current snapshot, changes it during the functional probe, or the shared checker fails, RSCTF writes one field-wide void instead. A void changes no team's denominator and never reuses the preceding tick.

4.6 Multiple hills and a short final epoch

Suppose one hill has weight **1.2** and local score **33.43**, while another has weight **0.8** and local score **70**:

$$E = \frac{(1.2)(33.43) + (0.8)(70)}{1.2 + 0.8} = 48.06.$$

Suppose two complete epochs score **80** and **60**, while a three-of-twelve-tick final epoch scores **40**. Its weight is **0.25**:

$$T = \frac{(1)(80) + (1)(60) + (0.25)(40)}{1 + 1 + 0.25} = 66.67.$$

5. FAILURE ADJUDICATION AND RECOVERY

5.1 Functional verdicts

Verdict or condition	Marker treatment	API arena treatment
Ok , stable current evidence	control/responsibility; may advance confirmation	score every eligible team from dense current evidence
Mumble or Offline	responsible team receives unhealthy evidence; confirmation breaks	field-wide void because the shared arena is not trustworthy
InternalError	field-wide void	field-wide void

Verdict or condition	Marker treatment	API arena treatment
missing/changing marker or snapshot	no new marker election according to exact claim rules	field-wide void; stale carry
reset/readiness/incomplete issuance	field-wide void	field-wide void
omitted eligible API team	not applicable	explicit zero in otherwise valid tick

5.2 Transaction and replica safety

The round engine uses a database advisory lock per game/hill plus a local single-flight guard. Unique keys allow one control result per game/challenge/round and one API score row per game/challenge/round/participation. Upserts are atomic; the implementation does not perform a read-check-then-insert sequence.

Before writing API scores, the checker takes a shared lock on the exact snapshot row whose digest bracketed the functional probe. A referee update cannot replace that snapshot until the score transaction completes.

5.3 Recovery

The admin console exposes stored cycle phase, old and replacement container identities, readiness failures, audit receipts, referee credential status, and last accepted snapshot time. **Retry** resumes the same idempotent lifecycle; it does not create a separate repair process.

6. SECURITY AND ANTI-CHEAT PROPERTIES

6.1 What the platform prevents

The API arena rejects or neutralizes:

- arbitrary external points, negative scores, and organizer-selected formula coefficients;
- raw team IDs and raw bearer capabilities in the evidence payload;
- malformed ratios, zero denominators, over-earned evidence, oversized budgets, duplicate token hashes, excessive objective count, oversized bodies, and rosters over 2,000 teams;
- stale round, cycle, reset, target, or container evidence;
- old capability hashes after issuance rotation;
- accepted-signature replay and out-of-order snapshots;
- post-deadline submissions;
- score carry for omitted teams;
- temporal mixing of nonlinear evidence; and
- partial feed scoring in the bundled referee when its cursor has a retention gap.

These controls constrain the platform/referee protocol. They cannot prove that an organizer-defined objective measures the intended skill.

6.2 What a challenge must do

A defensible API arena counts only meaningful verified actions. It should use unpredictable, expiring, one-use server tasks; bind each event to the capability hash that started it; include failed attempts in

integrity; publish fixed denominators; rate-limit per identity, client, and service; bound sessions and event retention; and expose an ordered evidence cursor.

Objectives should be conceptually distinct. Because RSCTF normalizes each component equally, repeating the same component to create hidden weight violates the declared scoring design. RSCTF requires one common objective-component count across a snapshot; organizers should keep those components' meaning and order fixed throughout the event.

6.3 Trust boundary and limitations

The referee is trusted infrastructure. HMAC proves that an accepted body came from a holder of the secret and was not altered in transit; it does not prove that the referee measured the challenge honestly. Run it outside the attacker-controlled application under a dedicated identity, protect its state and time source, and retain its audit logs through appeals.

Automation is permitted. RSCTF does not attempt to infer whether a person, script, or assisted workflow produced an event. All methods remain subject to the published challenge rules, credentials, scope, evidence checks, and rate limits.

7. PLAYER AND ORGANIZER OPERATIONS

7.1 Player procedure

For both formats:

1. fetch the current capability and exact hill state;
2. use only the player-facing challenge mechanic;
3. discard the old capability after every reset;
4. protect capabilities from logs, screenshots, and public repositories; and
5. verify the scoreboard rather than assuming an action was sampled.

For marker hills, keep the exact claim stable and the checked service healthy through confirmation. For API arenas, complete distinct verified actions, balance every objective, and avoid speculative requests that count as invalid attempts.

7.2 Organizer preflight

Before official scoring:

- freeze and publish the format, tick/epoch/cycle cadence, service weights, checker contract, API activity target, objective definitions, denominators, and appeal policy;
- confirm at least two accepted teams receive distinct current capabilities;
- run a full clean reset and reject the previous capability afterward;
- verify the independent functional checker before and after evidence reads;
- for marker mode, verify holder confirmation and enforceable cooldown;
- for API mode, verify HMAC signing, recognized/submitted equality, objective normalization, explicit omitted-team zero, referee restart, stale hash, replay, timestamp, feed-gap, and changing-snapshot behavior; and
- run at least one accelerated complete epoch with honest and invalid clients.

The official snapshot records roster, hills, images, formats, cadence, and weights. Later editor changes cannot rewrite finalized evidence.

7.3 During play and finalization

Monitor missing checker results, void reasons, reset phases, runtime identity, snapshot freshness, recognized-team counts, feed cursor lag, and referee errors. Pause scoring before rotating a referee secret, submit fresh current evidence, verify it, and then resume.

At the event deadline, reject late evidence, finish in-flight authoritative checks according to the deadline fence, finalize the proportional tail, and publish awards only from Settled results. Retain official configuration, capability issuance/revocation, immutable checker rows, API tick evidence, cycle receipts, and epoch rollups through the appeal period.

8. VALIDATION AND LIMITATIONS

The equations establish boundedness and several zero conditions. Unit and database tests establish implementation behavior for the tested cases. Neither kind of verification establishes competitive validity.

Organizers should measure:

- API activity and objective distributions;
- integrity loss and invalid-action patterns;
- missing/void tick frequency;
- referee lag, restart recovery, and feed retention margin;
- marker confirmation failures and control changes;
- repeat winners and cooldown use;
- readiness and reset latency; and
- score sensitivity to activity targets, challenge objectives, tick duration, cycle length, and epoch length.

A randomized checker sample cannot prove continuous state between observations. Capability hashing does not repair a low-entropy token source. A valid HMAC does not prove referee correctness. Equal objective normalization does not prove that the chosen objectives have equal difficulty. The formula cannot prevent collusion outside the evidence protocol. Because a shared arena outage is field-wide void evidence, the challenge must isolate and bound work per capability so one participant cannot turn resource exhaustion into a scoring veto. These are operational and challenge-design responsibilities.

9. CONCLUSION

RSCTF does not treat API KotH as boot2root KotH with a different transport. Marker hills remain exclusive contests scored by qualified acquisition, sustained control, and service reliability. API arenas let every team score concurrently from platform-normalized activity, objective, and integrity evidence.

The API formula uses a fixed weighted harmonic mean so play and performance are both necessary, then applies same-tick integrity. Independent normalization prevents native numeric scale from becoming accidental weight. Per-tick calculation prevents temporal recombination. Dense zero rows prevent stale team scores, while field-wide voids prevent infrastructure failures from becoming participant penalties.

Both formats retain exact identity fences, independent health checking, pristine lifecycle resets, fixed-ceiling hill aggregation, and finalized epoch settlement. These mechanisms make the result

reproducible and auditable. Fair competition still depends on the published rules, checker coverage, objective design, trusted referee operation, network equality, and incident review.

APPENDIX A. FREQUENTLY ASKED QUESTIONS

A.1 Is API mode just an observer for /koth/king ?

No. Marker mode has one holder and **A/C/R** evidence. API mode has concurrent teams and **E/P/I** evidence. The lifecycle infrastructure is shared, but the scoring semantics are different.

A.2 Can a referee submit points?

No. The wire contract accepts bounded integer evidence ratios and a current capability hash. RSCTF performs normalization and scoring.

A.3 Why use a harmonic mean?

It returns zero when either required play channel is zero and is sensitive to a weak channel. The fixed 35/65 weights give objective performance greater influence without allowing it to replace activity.

A.4 Why normalize objectives separately?

Raw sums make the largest native number dominate. Independent ratios let different units share the same **[0,1]** range before an equal-weight mean.

A.5 What happens when my team is missing from a valid API snapshot?

RSCTF writes an explicit zero row for that tick. It never repeats your previous score.

A.6 What happens when the whole snapshot or application is unavailable?

The tick is field-wide void and enters no team's denominator. This differs from one omitted team inside a valid snapshot.

A.7 Does a crown reset erase settled evidence?

No. It removes transient containers, sessions, claims, and active capabilities. Immutable checker evidence and finalized rollups remain.

A.8 Can one capability work on another hill or reset?

No. Capabilities and their hashes are resolved within an exact hill, target, cycle, reset attempt, and container generation.

A.9 Is there score versioning?

No. Marker and API are distinct formats, each with one constant formula. A migration voids pre-arena API holder evidence rather than interpreting it under the new arena formula.

A.10 Can automation participate?

Yes, subject to event rules. RSCTF validates evidence and scope rather than classifying a participant as human or automated.

APPENDIX B. NOMENCLATURE

Symbol or term	Definition
A, C, R	marker acquisition, control, and reliability rates
E, P, I	API activity, objective, and integrity rates for one tick
B^M	marker acquisition/control core
B^A	API weighted harmonic core for one tick

Symbol or term	Definition
G	integrity-adjusted API tick score rate
L^M, L^A	local marker/API hill score in [0,100]
w_h	frozen hill weight in [0.8,1.2]
z_{he}	field evidence switch for one hill and epoch
q_e	complete or partial epoch weight
Provisional	marker capability observed but not yet confirmed
Settled	official value using finalized epochs
Projected	informational value that also includes open evidence
Field void	sample excluded from every team's evidence
Explicit zero	one omitted API team's row in an otherwise valid tick

APPENDIX C. IMPLEMENTATION TRACEABILITY

Paths are relative to the repository revision containing this handbook.

Responsibility	Source of truth
Official format, roster, hill, cadence, and weight snapshot	src/services/ad/engine/koth_cycle/config.rs
Durable crown reset lifecycle	src/services/ad/engine/koth_cycle/lifecycle/
Marker claim transition and acquisition	src/services/ad/engine/koth_cycle/claims.rs
Exact marker read	src/services/ad/engine/koth_marker.rs
Signed API context, credential, HMAC, replay, and submission	src/controllers/game/koth/api/ , api_contract.rs
Stable exact-tick API snapshot read and tick formula	src/services/ad/engine/koth_api.rs
Marker/API checker persistence and dense zero rows	src/services/ad/engine/checker/koth.rs , checker/koth_api.rs
Pure marker and API epoch scoring	src/controllers/game/koth/scoring_formula.rs
SQL evidence aggregation	src/controllers/game/koth/scoring/evidence.rs

Responsibility	Source of truth
Finalized rollups	<code>src/controllers/game/koth/scoring/rollup/</code>
Board labels, format metadata, and ranking	<code>src/controllers/game/koth/board.rs</code> , <code>web/src/components/KothScoreboardTable.tsx</code>
Crown and constant marker schema	<code>src/migrations/m0046_koth_crown_cycles.rs</code> through <code>m0058_constant_koth_scoring.rs</code>
Referee credential/replay schema	<code>src/migrations/m0083_koth_api_observers.rs</code>
Challenge-owned objective scheme, normalized API snapshot, and score schema	<code>src/migrations/m0084_koth_api_arena.rs</code>

C.1 Core HTTP surface

Method and route	Purpose
GET <code>/api/game/{id}/ad/koth/{challengeId}/token</code>	caller's current exact-hill capability
GET <code>/api/game/{id}/ad/koth/{challengeId}/state</code>	hill lifecycle and marker holder state
GET <code>/api/game/{id}/ad/koth/scoreboard</code>	source-aware metrics, Projected, Settled, and ranks
GET <code>/api/game/{id}/ad/koth/timeline</code>	cumulative finalized/projected history
GET <code>/api/edit/games/{id}/ad/koth/state</code>	operator lifecycle and evidence view
POST <code>/api/edit/games/{id}/ad/koth/{challengeId}/recover</code>	idempotent lifecycle recovery
GET/POST/DELETE <code>/api/edit/games/{id}/ad/koth/{challengeId}/observer</code>	read metadata, create/rotate once, or revoke referee credential

Method and route	Purpose
GET <code>/api/v1/koth/games/{id}/challenges/{challengeId}/context</code>	exact round/runtime fence and eligible capability hashes
POST <code>/api/v1/koth/games/{id}/challenges/{challengeId}/observations</code>	signed bounded evidence snapshot; never points

Wire models use camelCase and Unix-millisecond timestamps. Enums are strings unless the platform's documented global exception applies.

C.2 Verification scope

The suite covers formula bounds and zero conditions, independent objective normalization, same-tick integrity, malformed evidence, HMAC scope, clock skew, replay, context rotation, current capability-hash resolution, unknown hashes, dense omitted-team zeros, stable snapshot bracketing, marker confirmation, personal cooldown denominators, tied champions, partial epochs, rollups, and ordinal ranks. The example repository additionally exercises one-use play, invalid attempts, token hashing, evidence pagination, persistent referee restart, feed-gap failure, and exact signed bodies.

The JavaScript lifecycle harness exercises current and stale capabilities, concurrent polling, cycle replacement, checker evidence, BYOC tunnels, and duplicate/integrity queries against the composed platform. Performance results belong in `tests/load/REPORT.md`; this handbook does not convert a benchmark into a fairness claim.

REFERENCES

1. RSCTF Project, “King of the Hill implementation,” repository-local source artifact, fixed marker and API-arena formulas, verified 28 July 2026.
2. K. Bock, G. Hughey, and D. Levin, “King of the Hill: A Novel Cybersecurity Competition for Teaching Penetration Testing,” in *Proceedings of the 2018 USENIX Workshop on Advances in Security Education*, Baltimore, MD, USA, 2018. [Online]. Available: <https://www.usenix.org/conference/ase18/presentation/bock>. Accessed: 28 July 2026.
3. CTFd, “King of the Hill,” *CTFd Documentation*, 2026. [Online]. Available: <https://docs.ctfd.io/docs/custom-challenges/king-of-the-hill/>. Accessed: 28 July 2026.
4. FAU Security Team, “Rules,” *FAUST CTF 2025*, 2025. [Online]. Available: <https://2025.faustctf.net/information/rules/>. Accessed: 28 July 2026.
5. OtterSec, “Submit dynamic scores,” *rCTF Documentation*, 2026. [Online]. Available: <https://github.com/ottersec/rctf/blob/main/apps/docs/src/content/docs/api/challenges/submit-dynamic-scores.md>. Accessed: 28 July 2026.